

Project 1 (v3): Guardrail & Evaluation Layer for Agentic Systems

1. Problem Statement

Production agent teams cannot ship on "it worked in my demo." This project builds an evaluation and guardrail harness that inspects an agent's execution trace and reports **evaluation signals with known limitations** — not a single objective score — covering task correctness, factual groundedness, and security/safety behavior.

2. Phase 1 Scope (Narrowed)

Target: single-agent, tool-using architectures only (this includes your own Code-Review Agent, which serves as the first real test subject).

Framework strategy: define one canonical **trace schema** — a JSON structure capturing step sequence, tool calls with arguments, per-step inputs/outputs, retrieved context, and timestamps. Each framework's adapter only has to translate its native run object into this schema.

- Adapter 1 (build first): **LangGraph**
- Adapter 2 (build second, to prove the schema generalizes): CrewAI or Claude Agent SDK

Sequencing: built horizontally (one full sub-phase at a time — §8), not as a thin vertical slice across all layers. A vertical slice reduces integration risk but costs more early cognitive load when several layers (trace design, eval design, security testing) are all being learned for the first time; a horizontal build lets each layer be understood in depth before moving to the next. To catch integration mismatches earlier without the full cost of a vertical slice, a short informal integration check ("does the trace schema actually contain every field the next phase needs?") is run at the end of 1a and again after 1c, before continuing.

3. Cross-Cutting Concerns (apply from 1a onward, not bolted on later)

Two concerns touch every sub-phase from the start rather than living in one phase, because retrofitting either one later is more expensive than building it in from day one:

3.1 Provider Abstraction + Mock Evaluator

Every LLM call (agent reasoning, judge scoring, claim extraction, entailment checking) goes through a single thin interface — e.g. `call_llm(provider, prompt, ...)` — rather than being hardcoded per call site. This is not a plugin architecture; it's a one-function wrapper. It matters because:

- You're already splitting calls across Groq and Gemini for quota reasons (§12) — without an abstraction, every provider swap touches every call site individually.
- A **mock evaluator** (a deterministic, fake "LLM" that returns fixed or rule-based responses) plugs into the same interface, letting you test the harness's logic — schema correctness, aggregation, dashboard rendering — without burning API quota or fighting real LLM non-determinism during development.

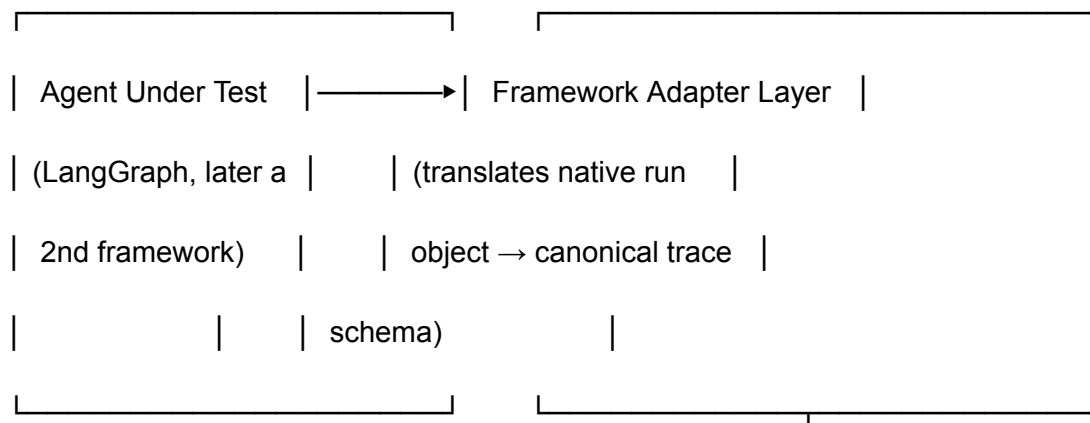
3.2 Reproducibility: Caching, Retries, Rate-Limit Handling

Given your free-tier constraints (Groq ~30 req/min, Gemini's daily caps), this is not optional polish:

- **Caching**: identical (prompt, model, version) calls are cached, so re-running the same eval suite during iteration doesn't re-spend quota on unchanged inputs.
- **Retries with backoff**: transient failures and rate-limit responses are retried with exponential backoff rather than failing the whole run.
- **Rate-limit awareness**: the provider abstraction (§3.1) tracks call volume per provider and can pace requests or fail gracefully rather than silently hitting a wall mid-run.

Skipping this doesn't remove the work — it converts "build it once, deliberately" into "debug mysterious rate-limit failures repeatedly during development."

4. Architecture



|



| Privacy/Redaction Filter | (§5)

| (PII/secret scrub before |

| persistence) |

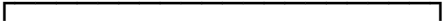


| Canonical Trace Store |

| (Postgres) |

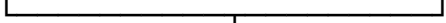
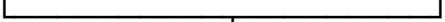


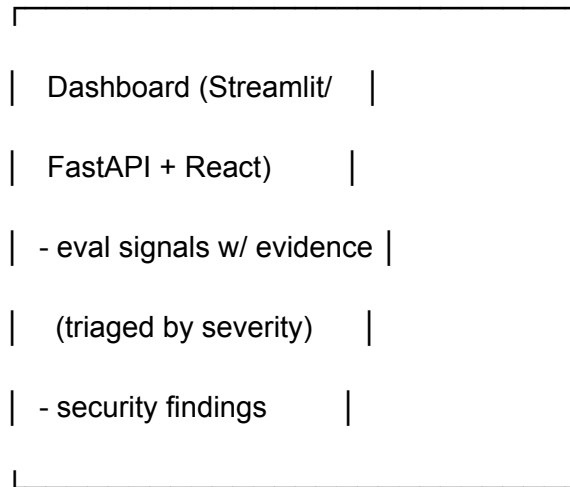
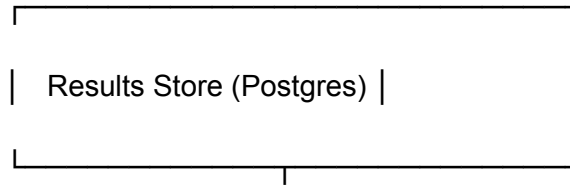
|



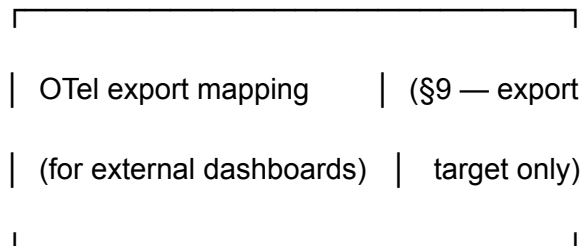
| Evaluation Engine | | Security Eval Module | | Telemetry Layer |

| (see §6) | | (see §7) | | (see §9, deferred) |





▼ (optional)



5. Privacy & Trace Redaction

Traces will contain full tool arguments, retrieved context, and — for the Code-Review Agent — potentially proprietary code. The security module's adversarial test suite also deliberately contains injection payloads. Storing all of this unredacted is a real gap, not a nice-to-have:

- A redaction filter runs **before** any trace is persisted (not after) — a basic PII/secret regex scrub (API keys, emails, tokens matching common secret patterns) is sufficient for project scope; enterprise-grade DLP is out of scope.
- This has to be decided at 1a, since retrofitting redaction after traces are already stored means reprocessing or wiping existing data.

6. Evaluation Engine

6.1 Structured Eval-Signal Schema

Every evaluation produces a structured object, not a bare score:

Field	Purpose
<code>signal_id</code>	Stable identifier for the evaluation result — lets a finding be referenced, tracked across re-runs, and linked to from the dashboard or a bug report
<code>score</code>	Numeric/categorical judgment
<code>rubric_breakdown</code>	Sub-scores per criterion (relevance, completeness, safety)
<code>evidence</code>	Exact spans/citations used to justify the score
<code>trace_span_ids</code>	Exact trace events (from the canonical trace schema, §2) that support the finding — lets a reviewer jump straight from a flagged signal to the precise agent step that caused it
<code>confidence</code>	Self-reported or derived (e.g. agreement across repeated judge calls)
<code>severity</code>	informational / low / medium / high / critical — lets the dashboard triage findings instead of presenting every flagged issue as equally urgent
<code>judge_model</code> + <code>prompt_version</code>	Which model/prompt produced this judgment
<code>evaluator_version</code>	Version of the harness's own scoring/aggregation logic — distinct from <code>prompt_version</code> . If the aggregation

Field	Purpose
	method changes later, old and new scores are only comparable if this is tracked
<code>eval_method</code>	rule-based / embedding / LLM-judge / human
<code>timestamp</code>	When the evaluation ran
<code>known_limitations</code>	Free-text note on this judge's failure modes

6.2 Claim-Level Groundedness (replaces embedding-similarity)

Embedding similarity conflates "sounds similar" with "is factually supported." Instead:

1. **Claim extraction** — LLM decomposes the answer into atomic factual claims
2. **Evidence retrieval** — for each claim, retrieve the specific supporting span from retrieved context
3. **Entailment check** — per (claim, evidence) pair: entailed / contradicted / not-enough-info
4. **Aggregate and report** — a single ratio (entailed / total claims) hides too much; report the full breakdown instead:

Metric	What it captures
<code>num_claims</code>	Total atomic claims extracted from the answer
<code>num_unsupported_claims</code>	Claims marked not-enough-info (evidence exists but doesn't confirm or deny)
<code>num_contradicted_claims</code>	Claims the evidence actively contradicts — the most severe failure mode
<code>evidence_coverage</code>	Fraction of claims for which a candidate supporting span was retrieved at all, before entailment is even checked
<code>retrieval_failure_rate</code>	Fraction of claims for which retrieval returned nothing relevant — separates "the answer is wrong" from "the retriever failed," which need different fixes
<code>claim_extraction_confidence</code>	The extraction step's own confidence that it correctly decomposed the answer — a noisy extraction undermines every downstream

Metric	What it captures
	number, so it needs to be visible, not assumed reliable

Reporting these separately (rather than collapsing to one groundedness score) is what makes the signal debuggable — a low score alone doesn't tell you whether the agent hallucinated, the retriever underperformed, or the claim extractor itself was unreliable.

6.3 Evaluator Types

Type	Use case
Rule-based / exact-match	Deterministic checks (format, required fields)
Structured validators	Schema validation on tool arguments and outputs
Embedding metrics	Cheap first-pass similarity signal (not the sole groundedness check)
LLM-as-judge	Task completion, rubric scoring
Mock evaluator	Deterministic fake responses for testing the harness itself (§3.1)
Human review	Spot-checks and calibration benchmark

6.4 Dataset Versioning

Test cases, expected outputs, metadata (difficulty, task type), and dataset version are stored explicitly — so a score is always reproducible against a known dataset snapshot.

7. Security Evaluation Module

Regex/keyword checks are a first layer only. Phase 1 adds a dedicated security-eval suite, framed against the **OWASP Top 10 for Agentic Applications 2026** (ASI01–ASI10, the current taxonomy specifically for autonomous agent systems — tool misuse, excessive agency, goal hijacking, etc.), which extends the separately-maintained **OWASP GenAI LLM Top 10 2026** (prompt injection, sensitive information disclosure, unbounded consumption, etc.). Both are cross-mapped to NIST's AI Risk Management Framework, which is used here as the governance rationale (continuous testing, documented mitigations) rather than a technical checklist.

Check	Method
Prompt injection (direct)	Run a known payload library against the agent, check for compliance
Indirect prompt injection	Plant an instruction inside a test document in retrieved context, check if the agent obeys it
Unauthorized tool calls / permission evaluation	Define an allow-list per agent role; flag any call outside it
Tool argument manipulation	Validate tool call arguments against an expected schema
Infinite loops / excessive retries	Step-count and repeat-detection thresholds
Excessive token/API usage	Simple per-run counters against a defined budget (full telemetry not required for this check — see §9)

7.1 Security Test Registry

Adversarial payloads (injection strings, planted document instructions, out-of-scope tool-call attempts) are stored the same way eval datasets are (§6.4) — with IDs and versions, not as a loose inline list. This matters because without versioning, a change in the security score over time is ambiguous: did the agent get worse, or did the payload set change? Extending the same dataset-versioning discipline to security payloads removes that ambiguity.

Deferred to future work (noted, not built): data exfiltration detection, cross-user/session leakage, multi-agent coordination checks, full insecure-output-handling coverage.

8. Phase 1 Is Too Large For One Cycle — Split Into Sub-Phases

Taken as one block, Phase 1 (trace schema + adapter + structured eval schema + claim-level groundedness + dataset versioning + evaluator types + security suite) has enough independently-tricky pieces that building it as a single unit risks ending up with several half-finished parts and no single milestone you can point to and say "this reliably works." The fix isn't to cut scope — it's to sequence it as separate, individually-benchmarkable milestones, where each one ships with its own test set and pass/fail bar before the next begins.

Sub-phase	Scope	Benchmark that proves it works
1a — Trace skeleton	Canonical trace schema + LangGraph adapter + privacy/redaction filter (§5) + Postgres trace store + provider abstraction & mock evaluator (§3.1) + caching/retry handling (§3.2)	Run 10 known agent executions through the adapter; every step, tool call, and I/O field round-trips into the schema with zero data loss; secrets/PII in test traces are confirmed redacted before storage; a repeated identical eval call is served from cache, not re-sent
<i>(informal integration check)</i>	—	Quick pass: does the stored trace actually contain every field 1b–1d will need?
1b — Basic evaluation	Structured eval-signal schema (§6.1, incl. <code>evaluator_version</code>) + rule-based and embedding evaluators + dataset versioning	A 20–30 case hand-labeled dataset; rule-based checks match expected pass/fail exactly, embedding scores correlate directionally with human judgment
1c — Groundedness	Claim extraction → evidence retrieval → entailment pipeline + the full metric set (§6.2)	On a small labeled set of (answer, known-correct/incorrect claims) pairs, the pipeline's entailed/contradicted/unsupported labels match human labels above an agreed threshold (e.g. 80%)
<i>(informal integration check)</i>	—	Quick pass: do groundedness results flow into the eval-signal schema correctly, with evidence and severity populated?
1d — Security suite	Prompt injection, indirect injection, permission eval, tool-arg validation, loop/retry	A fixed adversarial test suite (known injection payloads, a planted malicious instruction

Sub-phase	Scope	Benchmark that proves it works
	detection + versioned payload registry (§7.1)	in a test document, an out-of-scope tool call); the module catches a defined minimum percentage of them
1e — Dashboard	Surfaces 1a–1d output: eval signals with evidence/severity, security findings	Manual walkthrough: every flagged finding in the UI is traceable back to its <code>trace_span_ids</code>

Each sub-phase is a real stopping point — if you run out of time after 1c, you still have a working, benchmarked eval tool (trace capture + scoring + groundedness), not a pile of unfinished pieces. 1d and 1e depend on 1a–1c but not on each other, so they can also be reordered if security matters more to you than the dashboard, or vice versa.

9. Telemetry Layer (*not in Phase 1 — deferred*)

What it is: continuous, automatic collection of runtime health signals (latency per step, token/API usage, error rates) — the agent-system equivalent of a car's dashboard, as distinct from evaluation (which is a periodic "inspection report" on output quality).

Integration (when built): the canonical trace schema (§2) — not OpenTelemetry — remains the system's source of truth. OpenTelemetry is treated purely as an **export/mapping target**: an optional layer that translates your own trace data into OTel spans for interoperability with external dashboards, if needed. This is a deliberate choice, not a simplification — OTel's GenAI-specific semantic conventions are still under active development as of 2026, so building your core system's data model around them would mean inheriting an unstable spec. Your own schema stays stable regardless of how OTel's conventions evolve.

Why it's deferred rather than core: it doesn't feed the eval-signal or security modules in a way either depends on (the one exception — excessive-token-usage checks — runs on simple counters in Phase 1 without full OpenTelemetry instrumentation). Adding it later is additive, not a redesign, so there's no cost to deferring it.

10. Agent-Specific Evaluation Features (Phase 1 additions)

Feature	What it checks
Tool-call correctness	Right tool selected, valid arguments supplied

Feature	What it checks
Termination checks	Detects loops, runaway planning, excessive retries
Trajectory evaluation (<i>stretch</i>)	Judges the action sequence, not just the final answer
Recovery evaluation (<i>stretch</i>)	Tests agent behavior when a tool fails or returns malformed data

11. Feature Priority (beyond the sub-phased Phase 1 above)

Priority	Included
Phase 1 (sub-phased above)	Everything in 1a–1e
Should-have (Phase 2, <i>stretch</i>)	Telemetry integration (§9) · trajectory evaluation · second framework adapter · slice-based evaluation (by task type/difficulty) · evaluator calibration vs. hand-labeled benchmark · recovery evaluation
Could-have (future work, mention only)	Pairwise A/B agent comparison · inter-rater agreement across judges · confidence intervals · adversarial test-case auto-generation · human feedback loop · multi-agent coordination checks · data-exfiltration/cross-session monitoring

12. Tech Stack & Cost

Component	Tool	Cost
Primary LLM (agent + reasoning)	Groq (LLaMA 3.3 70B / 3.1 8B)	Free tier
Judge/entailment LLM pool	Google AI Studio (Gemini Flash) — separate quota pool to avoid contention with agent-run calls	Free tier

Component	Tool	Cost
Overflow	OpenRouter / Cloudflare Workers AI free models	Free tier
Embeddings	Local sentence-transformers model or free-tier <code>text-embedding-004</code>	Free
Database	Supabase or Neon (Postgres)	Free tier
Hosting	Railway / Render / Fly.io	Free tier
Tracing	LangSmith	Free tier
Telemetry (<i>Phase 2, deferred</i>)	Self-hosted, exported to OTel format if needed	Free

Note: claim-level groundedness makes several LLM calls per evaluated answer (1 extraction + 1 per claim). The provider abstraction and caching layer (§3) exist specifically to keep this affordable on free tiers — split judge/entailment calls onto a separate provider (Gemini) from agent-run calls (Groq), and cache repeated calls during iteration.

13. Why This Differentiates You

- Presents evaluation as a **signal with documented limitations**, not an objective score — a level of rigor most student (and many production) eval tools skip.
- Claim-level groundedness is a genuine technical contribution, not a wrapper around embedding similarity.
- Security evaluation grounded in current OWASP/NIST framing, with a versioned payload registry, shows awareness of how agentic systems actually fail in production, not just whether they answer correctly.
- Privacy-conscious by design (redaction before persistence) rather than an afterthought — a detail most student projects miss entirely.
- Deliberately scoped (single-agent, one framework first) rather than an unbuildable "works for any agent" claim — showing engineering judgment as much as technical skill.
- Sequenced as independently-benchmarked sub-phases, with lightweight integration checks between them, rather than one large build or a full vertical-slice rewrite — balances integration risk against the realistic cost of learning several new layers at once.
- Built on infrastructure (provider abstraction, caching, retries) that treats free-tier API constraints as a real design input, not an afterthought — directly reflects the actual conditions the project will be built under.